

第三讲 MATLAB程序设计

(Chapter 3 MATLAB Programming)

3.1 M文件 (M-file)

3.2 程序控制结构 (Program Structure)

3.3 函数文件 (Function File)

3.4 程序调试 (Program Debugging)

3.1 M文件 (M-file)

3.1.1 M文件概述 (M-file Overview)

用MATLAB语言编写的程序，称为M文件。M文件可以根据调用方式的不同分为两类：命令文件(Script File)和函数文件(Function File)。

Example 3-1 分别建立命令文件和函数文件，将华氏温度f转换为摄氏温度c。

程序1，首先建立**命令文件**并以文件名f2c.m存盘：

```
clear;          %清除工作空间中的变量  
f=input('Input Fahrenheit temperature: ');  
c=5*(f-32)/9
```

然后在MATLAB的命令窗口中输入f2c，将会执行该命令文件，执行情况为：

```
Input Fahrenheit temperature: 73  
c =  
    22.7778
```

程序2，首先建立**函数文件**f2c.m:

```
function c=f2c(f)
```

```
c=5*(f-32)/9
```

然后在MATLAB的命令窗口调用该函数文件。

```
clear;
```

```
y=input('Input Fahrenheit temperature: ');
```

```
x=f2c(y)
```

输出情况为:

```
Input Fahrenheit temperature: 70
```

```
c =
```

```
21.1111
```

```
x =
```

```
21.1111
```

3.1.2 M文件的建立与打开 (Establishment and Opening of M-files)

M文件是一个文本文件，它可以用任何编辑程序来建立和编辑，而一般常用且最为方便的是使用MATLAB提供的文本编辑器。

1. 建立新的M文件 (Create a new M-file)

为建立新的M文件，启动MATLAB文本编辑器有3种方法：

- (1) **菜单操作**。从MATLAB主窗口的File菜单中选择New 菜单项，再选择M-file命令，屏幕上将出现MATLAB 文本编辑器窗口。
- (2) **命令操作**。在MATLAB命令窗口输入命令edit，启动MATLAB文本编辑器后，输入M文件的内容并存盘。

(3) **命令按钮操作**。单击MATLAB主窗口工具栏上的New M-File命令按钮，启动MATLAB文本编辑器后，输入M文件的内容并存盘。

2. 打开已有的M文件（ Open an existing M-file ）

打开已有的M文件，也有3种方法：

(1) **菜单操作**。从MATLAB主窗口的File菜单中选择Open命令，则屏幕出现Open对话框，在Open对话框中选中所需打开的M文件。在文档窗口可以对打开的M文件进行编辑修改，编辑完成后，将M文件存盘。

(2) **命令操作**。在MATLAB命令窗口输入命令：edit 文件名，则打开指定的M文件。

(3) **命令按钮操作**。单击MATLAB主窗口工具栏上的Open File命令按钮，再从弹出的对话框中选择所需打开的M文件。

3.2 程序控制结构（Program Structure）

3.2.1 顺序结构（Sequence Structure）

1. 数据的输入（Data Input）

从键盘输入数据，则可以使用input函数来进行，该函数的调用格式为：**A=input(提示信息, 选项)**

其中提示信息为一个字符串，用于提示用户输入什么样的数据。

如果在input函数调用时采用's'选项，则允许用户输入一个字符串。例如，想输入一个人的姓名，可采用命令：

```
xm=input('What's your name?','s');
```

2. 数据的输出（Data Output）

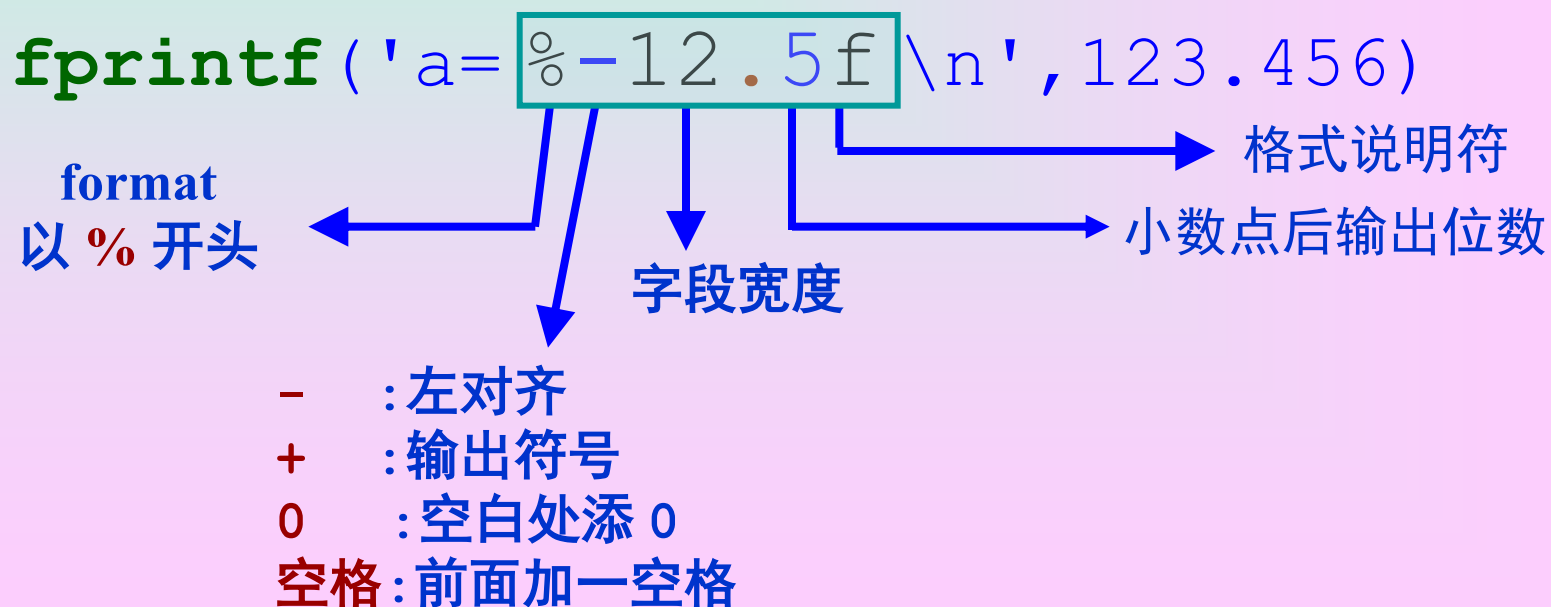
MATLAB提供的命令窗口输出函数主要有disp函数，其调用格式为：**disp(输出项)**

其中输出项既可以为字符串，也可以为矩阵（此时不显示矩阵的名字，输出格式更紧凑，且不留任何没有意义的空行）。

注意：（1）**disp**函数输出后默认换行；

（2）不支持格式化输出（如控制小数位数），如果需要格式化输出，可以使用 **fprintf** 函数；

（3）只能显示一个变量或表达式，如果需要同时显示多个变量，需拼接成字符串。



Example 3-2 输入x,y的值，并将它们的值互换后输出。
程序如下：

```
x=input('Input x please. ');  
y=input('Input y please. ');  
z=x;  
x=y;  
y=z;  
disp(x);  
disp(y);
```

Example 3-3 求一元二次方程 $ax^2 + bx + c = 0$ 的根。
程序如下：

```
a=input('a=');  
b=input('b=');  
c=input('c=');  
d=b*b-4*a*c;  
x=[(-b+sqrt(d))/(2*a),(-b-sqrt(d))/(2*a)];  
disp(['x1=',num2str(x(1)),',x2=',num2str(x(2))]);
```

3. 程序暂停 (Program Pause)

keyboard函数与**input**作用十分相似。当程序遇到此命令时，MATLAB就将暂停程序执行并处于键盘等待状态。处理完毕，键入**dbcont**程序继续运行，若键入**dbquit**则强制退出调试模式并终止程序。在M文件中使用此命令一般用于程序调试及修改变量。

pause函数用于使程序暂时中止运行，等待用户**按任意键后继续**运行。在程序调试及需要查看中间结果时十分有用。

调用格式：**pause(延迟秒数)**

如果省略延迟时间，直接使用**pause**，则将暂停程序，直到用户按任一键后程序继续执行。

若要**强行中止程序的运行**可使用**Ctrl+C**命令。

4.程序终止语句（Program termination statement）

调用格式：**return**

结束所在函数或脚本的执行，返回调用函数或指令窗

Example 3-4

```
for k = 1:3
    if k == 2
        disp(k)
        return
    end
    disp(k)
end
```

思考： 去掉**return**程序的运行结果是什么？

5. Matlab 的计时函数 (Timing Function)

调用格式: **tic, toc**

秒表计时, tic 开始, toc 结束

调用格式: **cputime**

返回开始时CPU时间(秒)

Example 3-5 计算两个 1000×1000 的随机矩阵相乘需要的时间

```
clc
```

```
clear all
```

```
n = 1000;
```

```
A = rand(n,n);
```

```
B = rand(n,n);
```

```
tic      %t=cputime
```

```
x = A*B;
```

```
t = toc  %t=cputime-t
```

3.3.2 选择结构 (Structure Selection)

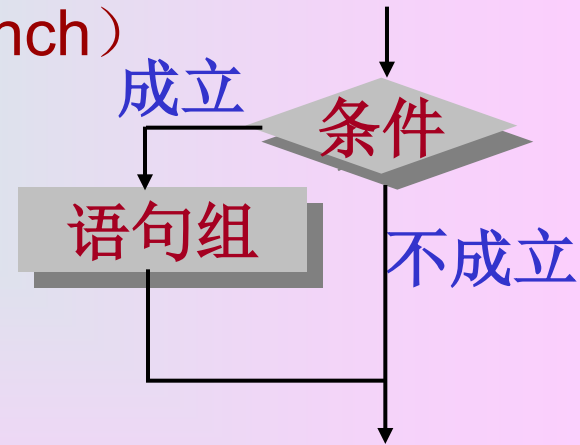
1 if语句 (if statement)

在MATLAB中，if 语句有3种格式。

(1) 单分支if语句 (Single branch)

语句格式：

```
if 条件  
    语句组  
end
```



当条件成立时，则执行语句组，执行完之后继续执行if语句的后继语句，若条件不成立，则直接执行if语句的后继语句。

(2) 双分支if语句 (Double branch)

语句格式:

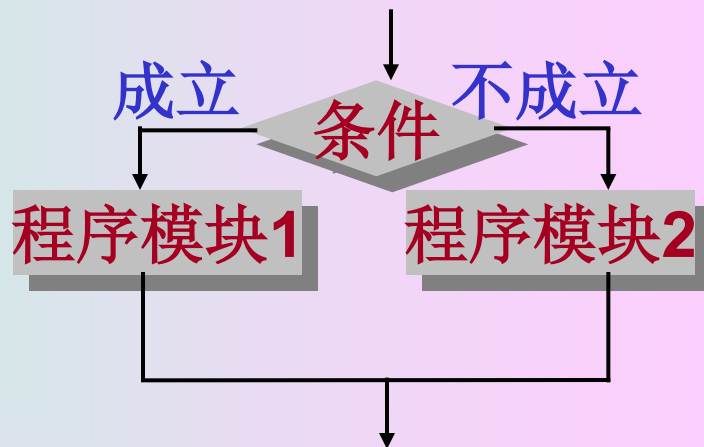
if 条件

语句组1

else

语句组2

end



当条件成立时，执行语句组1，否则执行语句组2，语句组1或语句组2执行后，再执行if语句的后继语句。

Example 3-6 计算分段函数值。

$$y = \begin{cases} \cos x + \sqrt{x^2 + 1} & (x = 10) \\ x\sqrt{x + \sqrt{x}} & (\text{其它}) \end{cases}$$

程序如下：

```
x=input('请输入x的值:'); %可以是向量
if x==10
    y=cos(x)+sqrt(x.*x+1);
else
    y=x.*sqrt(x+sqrt(x));
end
y
```

(3) 多分支if语句 (Multi branch)

语句格式:

```
if 条件1
    语句组1
elseif 条件2
    语句组2
.....
elseif 条件m
    语句组m
else
    语句组m+1
end
```

Example 3-7 将百分制的成绩转换为五级制的成绩输出。

```
clear
n=input('输入n= ');
if n>=90
    r='A'
elseif n>=80
    r='B'
elseif n>=70
    r='C'
elseif n>=60
    r='D'
else
    r='E'
end
```

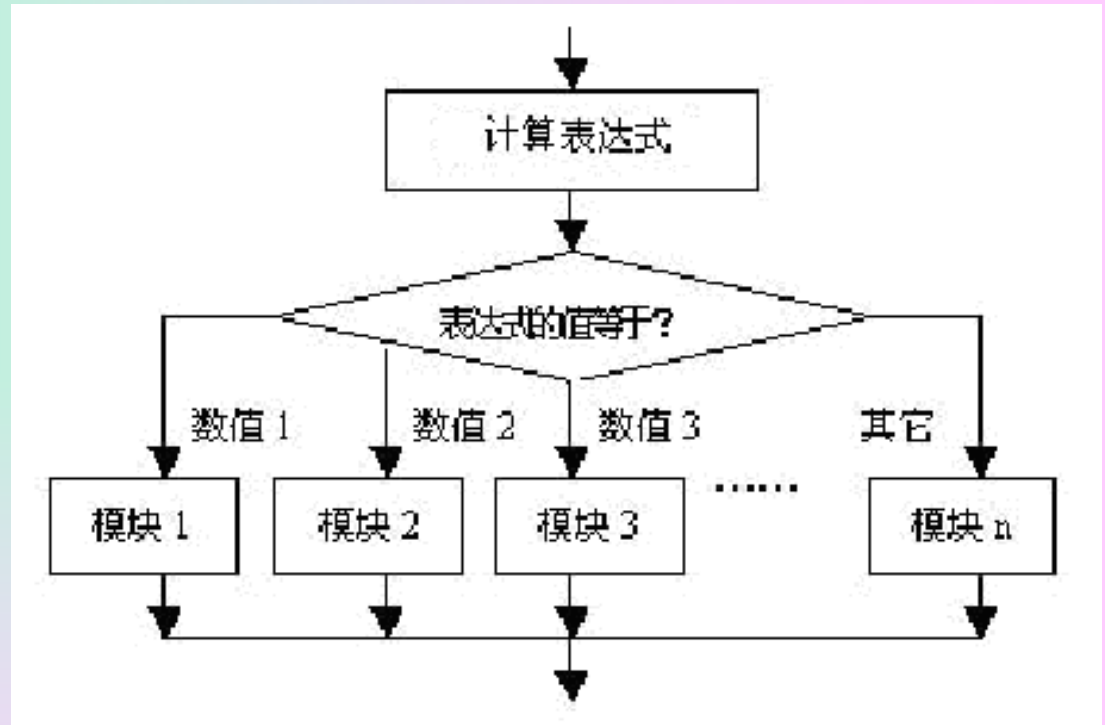
Example 3-8 输入一个字符，若为大写字母，则输出其对应的小写字母；若为小写字母，则输出其对应的大写字母；若为数字字符则输出其对应的数值，若为其他字符则原样输出。

```
c=input('请输入一个字符','s');  
if c>='A' & c<='Z'  
    disp(char(abs(c)+abs('a')-abs('A')));  
elseif c>='a' & c<='z'  
    disp(char(abs(c)-abs('a')+abs('A')));  
elseif c>='0' & c<='9'  
    disp(abs(c)-abs('0'));  
else  
    disp(c);  
end
```

2. switch语句（Switch Statement）

语句格式：

```
switch 表达式  
  case 表达式1  
    语句组1  
  case 表达式2  
    语句组2  
  .....  
  case 表达式m  
    语句组m  
  otherwise  
    语句组m+1  
end
```



说明：（1）**Switch**子句后面的表达式应为一个标量或一个字符串；

（2）**case**子句后面的值不仅可以为一个标量或一个字符串，而且还可以为一个单元矩阵。

（3）如果**case**子句后面的值为一个单元矩阵，则表达式的值等于该单元矩阵中的某个元素时，执行相应的语句组。

（4）**case**中有一个满足，就不再执行后面其他的**case**情形了。

Example 3-9 某商场对顾客所购买的商品实行打折销售，标准如下(商品价格用price来表示):

price<200 没有折扣

200≤price<500 3%折扣

500≤price<1000 5%折扣

1000≤price<2500 8%折扣

2500≤price<5000 10%折扣

5000≤price 14%折扣

输入所售商品的价格，求其实际销售价格。

```
price=input('请输入商品价格: ');
switch fix(price/100)
    case {0,1}          %价格小于200
        rate=0;
    case {2,3,4}        %价格大于等于200但小于500
        rate=3/100;
    case num2cell(5:9)   %价格大于等于500但小于1000
        rate=5/100;
    case num2cell(10:24) %价格大于等于1000但小于2500
        rate=8/100;
    case num2cell(25:49) %价格大于等于2500但小于5000
        rate=10/100;
    otherwise           %价格大于等于5000
        rate=14/100;
end
price=price*(1-rate)    %输出商品实际销售价格
```

3. try语句（try statement）

语句格式：

```
try  
    语句组1  
catch  
    语句组2  
end
```

try语句先试探性执行语句组1，如果语句组1在执行过程中出现错误，则将错误信息赋给保留的**lasterr**变量，并转去执行语句组2。这种试探性执行语句是其他高级语言所没有的。

Example 3-10 矩阵乘法运算要求两矩阵的维数相容，否则会出错。先求两矩阵的乘积，若出错，则自动转去求两矩阵的点乘。

程序如下：

```
A=[1,2,3;4,5,6]; B=[7,8,9;10,11,12];
```

```
try
```

```
    C=A*B;
```

```
catch
```

```
    C=A.*B;
```

```
end
```

```
C
```

```
lasterr
```

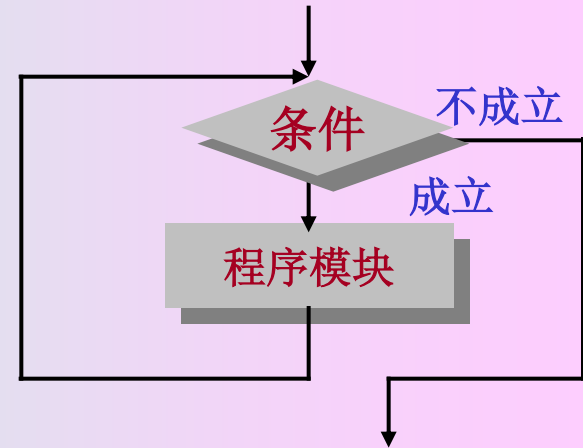
```
%显示出错原因
```

3.2.3 循环结构（Loop Structure）

1. for语句（for statement）

语句格式为：

```
for 循环变量=初值:步长:终值  
    循环体语句  
end
```



说明：（1）步长可为正、负值，默认为1，循环结束时循环变量取最后一个值（不一定是终值，如for k=1:3:11，循环结束时k的值为10）。

Example 3-11 使用for结构计算 $1+2+3+\dots+100$

```
clear  
sum=0;  
for i=1:100  
    sum=sum+i;  
end  
sum
```

(2) **for**语句针对循环变量（向量）的每一个元素执行一次循环体，循环次数就是向量中元素的个数。

```
for k=[1,3,2,5]
    k
end
```

(3) **for**语句中，表示循环次数的向量元素一旦确定将不会再改变。如果在**for**语句的表达式中含有变量，即便在循环体中改变变量的值，向量的元素也不会改变。

```
n=2;
for k=1:2:n+8
    n=5;
    k
end
```

Example 3-12 求定积分。

$$\int_0^{3\pi} e^{-\frac{x}{2}} \sin\left(x + \frac{\pi}{6}\right) dx$$

程序如下：

```
a=0;b=3*pi;n=1000; h=(b-a)/n;  
x=a:h:b; f=exp(-0.5*x).*sin(x+pi/6);  
s=0;  
for i=1:n  
    s=s+(f(i)+f(i+1))*h/2;  
end  
s
```

事实上，**MATLAB**提供了有关数值积分的标准函数，实际应用中可直接调用这些函数求数值积分。

Example 3-13

已知 $y = 1 + \frac{1}{2^2} + \frac{1}{3^2} + \cdots + \frac{1}{n^2}$ 当 $n=100$ 时，求 y 的值。

解：程序如下：

```
y=0;
```

```
n=100;
```

```
for i=1:n
```

```
    y=y+1/i/i;
```

```
end
```

```
y
```

在实际MATLAB编程中，采用循环语句会降低其执行速度，所以前面的程序通常由下面的程序来代替：

```
n=100;
```

```
i=1:n;
```

```
y=sum(1./i.^2);
```

```
y
```

for语句更一般的格式:

```
for 循环变量=矩阵表达式  
    循环体语句  
end
```

执行过程是依次将矩阵的**各列元素**赋给循环变量，然后执行循环体语句，直至各行元素处理完毕。

Example 3-14 已知5个学生4门功课的成绩，求每名学生的总成绩。

```
s=0;  
a=[65,76,56,78;98,83,74,85;76,67,78,79;98,58,42,73;67,  
89,76,87];  
for k=a  
    s=s+k  
end  
disp(s);
```

Experiment Content

Ex-1: 分别用**if**语句和**switch**语句实现以下计算，其中 a 、 b 、 c 的值从键盘输入。

$$\begin{cases} ax^2 + bx + c, & 0.5 \leq x < 1.5 \\ a \sin^c b + x, & 1.5 \leq x < 3.5 \\ \ln \left| b + \frac{c}{x} \right|, & 3.5 \leq x < 5.5 \end{cases}$$

Ex-2: 产生**20**个两位随机整数，输出其中小于平均值的偶数。(提示：平均值可用**mean**函数)

Ex-3: 输入**20**个数，求其中最大数和最小数。要求分别用循环结构和调用**MATLAB**的**max**函数、**min**函数来实现。

Experiment Content

Ex-4: 已知 $s = 1 + 2 + 2^2 + 2^3 + \cdots + 2^{63}$

分别用循环结构和调用**MATLAB**的**sum**函数求**s**的值。

Ex-5: 分析下列程序，并输出结果。

```
s=0;
a=[12,13,14;15,16,17;18,19,20;21,22,23];
for k=a
    for j=1:4
        if rem(k(j),2)~=0
            s=s+k(j);
        end
    end
end
s
```

2. while语句（while statement）

while语句的一般格式为：

```
while (条件)
    循环体语句
end
```

其执行过程为：若条件成立，则执行循环体语句，执行后再判断条件是否成立，如果不成立则跳出循环。

Example 3-15 使用**while**结构计算 $1+2+3+\dots+100$

```
clear
sum=0; i=0;
while i<100
    i=i+1;
    sum=sum+i;
end
sum
```

Example 3-16 一张纸厚0.06mm且足够大，试问将纸对折多少次，其厚度将超过10000 m?

```
h = 0.06e-3 %单位m
```

```
n = 0;
```

```
while h < 1e4
```

```
    n = n + 1;
```

```
    h = 2*h;
```

```
end
```

```
disp(['对折 ',num2str(n),' 次其厚度将超过10000米'])
```

Example 3-17 从键盘输入若干个数，当输入0时结束输入，求这些数的平均值和它们之和。

```
程序： sum=0;
        cnt=0;
        val=input('Enter a number (end in 0):');
while (val~=0)
    sum=sum+val;
    cnt=cnt+1;
    val=input('Enter a number (end in 0):');
end
if (cnt > 0)
    sum
    mean=sum/cnt
end
```

3. 流程控制语句（ Flow control statement ）

1. Break statement

break 语句用于立即跳出包含该语句的各种 while 和 for 循环语句结构。该语句一般与 if 语句配合使用。例如：

```
while <条件1>
```

```
.....  
    if <条件2>  
        break;
```

```
    end
```

```
.....  
end
```

2. Continue statement

continue 语句用于提前结束本次循环，可用于各种 while 和 for 循环语句结构。例如：

```
while <条件1>
```

```
.....  
    if <条件2>  
        continue;
```

```
    end
```

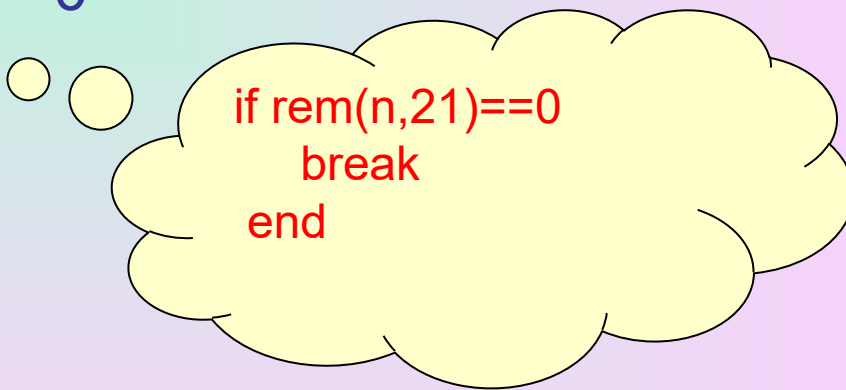
```
.....  
end
```

3. Return statement

return 语句终止当前命令的执行，并且返回到调用函数或者等待键盘输入指令。

Example 3-18 求[100,200]之间第一个能被21整除的整数。

```
clc
clear
for n=100:200
    if rem(n,21)~=0
        continue
    end
    break
end
n
```



```
if rem(n,21)==0
    break
end
```

4. 循环的嵌套（ Nesting of Loops ）

➤ 如果一个循环结构的循环体又包括一个循环结构，就称为循环的嵌套，或称为多重循环结构。可以按照嵌套层数，分别叫做二重循环、三重循环等。处于内部的循环叫作内循环，处于外部的循环叫作外循环。

➤ 在设计多重循环时，要特别注意内、外循环之间的关系，以及各语句放置的位置，不要搞错。

Example 3-19 求某自然数范围内的全部素数。

```
m=input('m='); %t=1;
for k=2:m;
    for p=2:k
        if mod(k,p)==0
            break
        end
    end
    if k==p
        k %s(t)=k; t=t+1;
    end
end %disp(['素数:',num2str(s)])
```

```
m=input('m=');
p=2:m;
for i=2:sqrt(m) %m-1
    n=find(mod(p,i)==0&p~=i);
    p(n)=[];
end
p
```

```
m=input('m=');
p=2:m;
p(isprime(p))
```

Example 3-20 若一个数等于它的各个真因子之和，则称该数为完数，如 $6=1+2+3$ ，所以6是完数。求[1,500]之间的全部完数。

```
for m=1:500
    s=0;
    for k=1:m/2
        if rem(m,k)==0
            s=s+k;
        end
    end
    if m==s
        disp(m);
    end
end
6 28 496
```

3.3 函数文件（Function File）

3.3.1 函数文件的基本结构（Basic Structure）

函数文件由function语句引导，其基本结构为：

function [输出形参表]=函数名(输入形参表)

注释说明部分

函数体语句

end

说明：（1）函数文件名与函数名一般应相同（当不同时，将忽略函数名而确认函数文件名）。

（2）注释说明部分：是对所写函数功能的一个简要描述，每行都应以前导%引导，其内容并不执行，只起注释作用。用户可以用“help 函数名”和“lookfor 函数名”命令显示出注释语句段的内容。

（3）当函数的返回值只有一个时，可以省去输出形参表外[]。

（4）关于return语句。执行到该语句就结束函数的执行，程序转至调用处。省略return语句被调函数执行完自动返回。

Example 3-21 编写函数文件求半径为 r 的圆的面积和周长。

函数文件如下：

```
function [s,p]=fcircle(r)
%CIRCLE calculate the area and perimeter of a circle of
radii r
```

```
%r      圆半径
%s      圆面积
%p      圆周长
```

```
%2013年11月30日编
```

```
s=pi*r*r;
```

```
p=2*pi*r;
```

将以上函数文件以文件名`fcircle.m`存入`c:\matlabr11\work`下，然后在MATLAB命令窗口调用该函数：

```
[s,p]=fcircle(10)
```

3.3.2 函数调用 (Function Call)

函数调用的一般格式是：

[输出实参表]=函数名(输入实参表)

注意：（1）函数调用时各实参出现的顺序、个数，应与函数定义时形参的顺序、个数一致，否则会出错。函数调用时，先将实参传递给相应的形参，从而实现参数传递，然后再执行函数的功能。

```
function [输出形参表]=函数名(输入形参表)
    注释说明部分
    函数体语句
end
```

Example 3-22 利用函数文件，实现直角坐标(x,y)与极坐标(ρ, θ)之间的转换。

Example 3-22 利用函数文件，实现直角坐标(x,y)与极坐标(ρ, θ)之间的转换。

函数文件**tran.m**:

```
function [rho,theta]=tran(x,y)
rho=sqrt(x*x+y*y);
theta=atan(y/x);
```

调用tran.m的命令文件main1.m:

```
x=input('Please input x=:');
y=input('Please input y=:');
[rho,the]=tran(x,y);
rho
the
```

(2) 在MATLAB中，函数可以嵌套调用，即一个函数可以调用别的函数，甚至调用它自身。一个函数调用它自身称为函数的递归调用。

Example 3-23 利用函数的递归调用，求 $n!$ 。
 $n!$ 本身就是以递归的形式定义的：
$$n! = \begin{cases} 1 & n \leq 1 \\ n(n-1)! & n > 1 \end{cases}$$

显然，求 $n!$ 需要求 $(n-1)!$ ，这时可采用递归调用。

递归调用函数文件**factor.m**如下：

```
function f=factor(n)
```

```
if n<=1
```

```
    f=1;
```

```
else
```

```
    f=factor(n-1)*n;    %递归调用求(n-1)!
```

```
end
```

在命令文件main2.m中调用函数文件factor.m，求
 $s=1! + 2! + 3! + 4! + 5!$ 。

```
s=0;
```

```
for i=1:5
```

```
    s=s+factor(i);
```

```
end
```

```
s
```

在命令行窗口运行命令文件，得到运行结果如下：

```
>>main2
```

```
s=
```

```
153
```

缺点：通常函数递归调用，使用内存大，程序运行速度慢。许多的递归程序，也可以用递推的形式来完成。

```
function y = njl(n)
% y = njl(n) 计算 n 的阶乘
% 输入参数 n 是自然数
% 输出参数 y 是 n 的阶乘
if (n < 0) | (floor(n) ~ = n)
    error('输入参数必须是自然数')
end
if n == 0
    y = 1;
elseif n == 1
    y = 1;
else
    y = 1;
    for k = 1:n
        y = y * k;
    end
end
```

% 如果输入参数不是自然数
% 则显示出错信息

% 如果输入参数是 0
% 0 的阶乘是 1
% 输入参数是 1
% 1 的阶乘是 1
% 如果是大于 1 的自然数

% 递推

3.3.3 函数所传递参数的可调性（ The tunability of the parameters ）

MATLAB在函数调用上与一般高级语言有一个不同之处，就是函数所传递参数数目的可调性。凭借这一点，一个函数可完成多种功能。

在调用函数时，MATLAB用两个预定义变量 **nargin** 和 **nargout** 分别记录调用该函数时的输入实参和输出实参的个数。只要在函数文件中包含这两个变量，就可以准确地知道该函数文件被调用时的输入输出参数个数，从而决定函数如何处理。

Example 3-24 nargin示例

函数文件 **examp.m**:

```
function fout=charray(a,b,c)
if nargin==1
    fout=a;
elseif nargin==2
    fout=a+b;
elseif nargin==3
    fout=(a*b*c)/2;
end
```

命令文件 **mydemo.m**:

```
x=[1:3];y=[1;2;3];
examp(x)
examp(x,y')
examp(x,y,3)
```

```
>> mydemo
```

```
ans =
```

```
1     2     3
```

```
ans =
```

```
2     4     6
```

```
ans =
```

```
21
```

```
function [x0, y0] = myplot(x, y, npts,  
angle, subdiv)  
% MYPLOT Plot a function.  
% MYPLOT(x, y, npts, angle, subdiv)  
% The first two input arguments are  
% required; the other three have  
% default values.  
  
...  
if nargin < 5, subdiv = 20; end  
if nargin < 4, angle = 10; end  
if nargin < 3, npts = 25; end  
  
...  
if nargout == 0  
plot(x, y)  
else  
x0 = x;  
y0 = y;  
end
```

Experiment Content

Ex-1: 编写一个函数文件，求小于任意自然数n的Fibonacci数列各项。Fibonacci数列定义如下：

$$\begin{cases} f_1 = 1, & n = 1 \\ f_2 = 1, & n = 2 \\ f_n = f_{n-1} + f_{n-2}, & n > 2 \end{cases}$$

求Fibonacci数列的第20项。

Ex-2: 建立5×6矩阵，要求输出矩阵第n行元素。当n值超过矩阵的行数时，自动转为输出矩阵最后一行元素，并给出出错信息。

Ex-3: 当n分别取100,1000,10000时, 求下列式子的值。要求分别用循环结构和向量运算(使用**sum**或**prod**函数)来实现。

$$(1) \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \cdots + \frac{1}{n^2} + \cdots \left(= \frac{\pi^2}{6} \right)$$

$$(2) \left(\frac{2 \times 2}{1 \times 3} \right) \left(\frac{4 \times 4}{3 \times 5} \right) \left(\frac{6 \times 6}{5 \times 7} \right) \cdots \left(\frac{(2n)(2n)}{(2n-1)(2n+1)} \right) \cdots \left(= \frac{\pi}{2} \right)$$

Ex-4: 已知

$$y = \frac{f(40)}{f(30) + f(20)}$$

(1) 当 $f(n) = n + 10 \ln(n^2 + 5)$ 时, y 的值是多少。

(2) 当 $f(n) = 1 \times 2 + 2 \times 3 + 3 \times 4 + \cdots + n \times (n+1)$ 时, y 的值是多少。

3.4 全局变量和局部变量 (Local and Global Variables)

根据作用域的不同，可以将程序中的变量分为：**局部变量**、**全局变量**。命令文件中的变量都是全局变量，作用域是整个**MATLAB**工作空间，即全程有效，所有的函数都可以对它进行存取和修改。而函数文件中的内部变量是局部变量，只能在本函数范围内使用。如要在函数中实现变量的函数间调用，则要使用全局变量。

全局变量的语句格式：**global 变量列表**

在定义和使用全局变量时要注意：

- 1) 全局变量列表中的各个变量名不能用逗号分隔；
- 2) 一般把全局变量的定义语句放在文件的前面，且多用大写字母；
- 3) 注意全局变量的副作用；
- 4) 必须逐个用**global**语句加以定义。

Example 3-25 全局变量应用示例。

先建立函数文件wadd.m——将输入的参数加权相加。

```
function f=wadd(x,y)
global ALPHA BETA
f=ALPHA*x+BETA*y;
```

在命令窗口中输入：

```
global ALPHA BETA
ALPHA=1;
BETA=2;
s=wadd(1,2)
```

3.5 函数句柄和内联函数

(Function handles and inline functions)

3.5.1 子函数 (Subfunction)

在MATLAB中，可以在一个函数文件中同时定义多个函数，其中函数文件中出现的第一个函数称为主函数（Primary Function），其他函数称为子函数（Subfunction）。

注意：（1）子函数只能由同一个函数文件中的函数调用；
（2）在保存函数文件时，函数文件名一般和主函数名相同，外部程序只能对主函数进行调用。
（3）同一函数文件中主函数和子函数的工作区是彼此独立的。

例如建立func.m文件：

```
function d=func(a,b,c)      %主函数
d=subfunc(a,b)+c;
function c=subfunc(a,b)    %子函数
c=a*b;
```

在命令行窗口调用主函数，结果如下：

```
>>func(3,4,5)
```

```
ans=
```

```
17
```

```
function [mean,stdev] = stats(x)
% [mean,stdev] = stats(x)计算输入向量的均值和均方差
% 输入参数 x 是向量
% 第一个输出参数 mean 是向量各元素的平均值
% 第二个输出参数 stdev 是向量的均方差
% 例如,取向量 x = [1,2,3,4,5]
% 调用 [m,s] = stats(x), 计算可得
% 均值 m = 3
% 均方差 s = 1.4142
% 程序完成时间: 2014 年 05 月 16 日

n = length(x); % 计算向量长度
mean = avg(x,n); % 求向量平均值
stdev = sqrt(sum((x-avg(x,n)).^2)/n); % 计算均方差


function mean = avg(x,n)
% 子函数 avg 计算向量的平均值
mean = sum(x)/n;
end
```

3.5.2 利用函数句柄执行函数（Function Handles）

1. 函数句柄的创建

语法：

```
h_fun=@fun           %创建函数句柄
```

说明：fun是函数名（可以是内部函数，也可以是编写的函数文件），h_fun是函数句柄。

Example 3-26 创建MATLAB内部函数的句柄。

```
h_sin=@sin;          %创建函数句柄
```

2. 使用函数句柄的优点

(1) 提高函数调用的速度

不使用函数句柄时，对函数的每次调用都要为该函数进行全面的路径搜索，直接影响了速度。

(2) 使函数调用象使用变量一样方便、简单。

(3) 可迅速获得同名**重载函数**的位置、类型信息。

3. 用feval命令执行函数

函数也可以使用feval命令直接执行，feval命令可以使用函数句柄或函数名。

语法：

```
[y1,y2,...]=feval(h_fun,arg1,arg2...)
```

```
[y1,y2,...]=feval('funname',arg1,arg2...)
```

说明：h_fun是函数句柄，funname是函数名，arg1、arg2...是输入参数，y1、y2...是输出参数。

比如，s=@sin

```
feval(s,pi/2)
```

```
feval('sin',pi/2)
```

3.5.3 内联函数 (Inline Function)

1. 内联函数的创建

语法: `inline('string',arg1,arg2,...)`

说明: `string`必须是不带赋值号(=)的字符串; `arg1`和`arg2`是函数的输入变量。**不用将其储存为一个M文件, 同时又可以像使用一般函数那样调用它。**

Example 3-27 创建内联函数

```
f=inline('sin(x)*exp(-z*x)','x','z')
```

2. 执行内联函数

内联函数的调用同标准函数一样, 如:

```
y=f(5,0.3)      %调用函数f
```

内联函数还可以直接使用`feval`命令执行。

语法: `[y1, y2,...]=feval(inline_fun,arg1,arg2...)`

```
y=feval(f,5,0.3)
```

3.5.4 匿名函数 (Anonymous Function)

和内联函数类似，可以让用户编写简单的函数而不需要创建M文件。因此，匿名函数具有inline函数的所有优点，并且效率比inline函数高，同时还具有一些独有的特点。

匿名函数很像C语言中的宏，但是它比宏的功能更加强大。语法：`fhandle=@(arglist) expression`

其中fhandle就是调用该函数的函数句柄，arglist是参数列表，多个参数使用逗号分隔，expression则是该函数的表达式，例如定义x和y的平方和函数：

```
>> f=@(x,y) x*x+y*y
```

一旦定义了匿名函数，使用方式和inline函数类似，例如计算f(3,4)：

```
>> f(3,4)
```

3.6 程序调试 (Program Debugging)

在编写程序过程中，难免会发生一些错误，要准确地找到错误并修改并非易事。虽然MATLAB语言比其它语言简单，但仍有自己的语法。对M文件而言，主要的错误类型有两类。

1. 语法错误 (Syntax Error)

MATLAB可以检查出大部分的语法错误，显示错误的信息并标出错误在程序中的位置。例如，函数名拼写错误、括号不配对等。

2. 运行错误 (Run-time Error)

这类错误的根源一般和程序本身有关，例如当出现溢出、死循环等情况时，将出现运行错误。

一般**MATLAB**在编译过程中就能发现语法错误，这类错误比较容易定位与更正，而程序在运行期间发现的错误却难以跟踪定位。虽然当程序运行期间发生错误时不会停止程序的执行，也不会显示错误位置，但无法得到正确结果。只有在程序执行结束或者出错而返回到基本工作间时，才知道发生了运行错误，而这时各函数的局部工作空间已关闭，致使错误难以跟踪，必须采取以下方法来查询错误：

1. 在运行期间可能发生的函数文件中，**删除某些命令末尾的分号**，这样可显示出一些中间计算结果，从中可能发现一些问题。

2. **使用MATLAB的调试工具**。调试器可以准确地找到运行错误。通过设置断点可以使程序运行到某行暂停，查看和修改工作空间的中间变量；也可以逐行运行程序，对流程进行完全监控。

3. 程序优化（ **program optimization** ）

（1）采用向量化运算。为了提高程序的执行速度，常用向量或矩阵运算来代替循环操作。

（2）预分配内存空间。通过在循环之前预分配向量或数组的内存空间可以调for循环的处理速度。

程序1:

```
clear;
```

```
a=0;
```

```
for n=2:1000
```

```
    a(n)=a(n-1)+10;
```

```
end
```

程序2:

```
clear;
```

```
    a=zeros(1,1000);
```

```
for n=2:1000
```

```
    a(n)=a(n-1)+10;
```

```
end
```

(3) 减小运算强度。尽量采用运算量更小的运算，从而提高运算速度。一般来说，乘法比乘方运算快，加法比乘法运算快。

例如：

```
clear;
```

```
a=rand(32);
```

```
x=a.^3;
```

```
y=a.*a.*a;           %花的时间更少
```